

Forsøk 1



Pågår

NESTE: Lever oppgave



Legg til kommentar

Ubegrenset antall forsøk tillatt

▼ Detaljer

Guidelines

To get credit for this exercise, you must:

- **Hand in the exercise on time—delayed hand ins will not receive any credit.**
- Hand in the Jupyter notebook file (*.ipynb) and the same file exported as PDF. If PDF export fails, hand in the IPython notebook. Without the PDF, I will not be able to give you feedback directly in the document.
- **You must include your names and exercise group number at the top of the notebook:**
 - **No name, no credit!**
 - **One point is deducted if the group number is missing.**
- Place the exercises in the order given below in the notebook.
- Begin each exercise with a second-level heading (##) reading "Exercise X".
- Make reasonable attempts at solving all exercises.

For an introduction to how to typeset mathematics in notebooks, see, e.g., the [LaTeX section](https://patrickwalls.github.io/mathematicalpython/jupyter/latex/)

(<https://patrickwalls.github.io/mathematicalpython/jupyter/latex/>) of [Mathematical Python](https://patrickwalls.github.io/mathematicalpython/)

(<https://patrickwalls.github.io/mathematicalpython/>). You are also encouraged to look at the Markdown and LaTeX code in the lecture notebooks.

Exercise 1 (4 points)

Use the master method to give tight asymptotic bounds for the following recurrences if the master method can be applied:

a) $T(n) = 2T(n/4) + 1$

b) $T(n) = 2T(n/4) + \sqrt{n}$

c) $T(n) = 2T(n/4) + n$

d) $T(n) = 2T(n/4) + n^2$

Exercise 2 (4 points)

The following algorithm to create random permutations of array A, but should **never** return the original ordering of A. The algorithm is similar to the one we investigated in class. But it is also correct, i.e., will it generate all possible permutations of A (except the original A itself) with equal probability? Assume that all elements of A are unique.

```
Permute-Without-Identity(A)
  n = A.length
  for i = 1 to n-1
    swap A[i] with A[Random(i+1, n)]
```

Hint: Begin by experimenting with small arrays A. No more than three elements are needed to see how the algorithm behaves.

Exercise 3 (2 points)

Using the figure illustrating the operation of heapsort in Lecture 6 as an example to illustrate how Heapsort operates on the array $A = \langle 6, 2, 12, 18, 7 \rangle$.

You can draw your solution with paper and pencil, take a photo and include it in your notebook as by inserting the following code in a text cell:

```
![SomeText](figure_filename_with_extension)
```

Exercise 4 (4 points)

Give an $O(n \lg k)$ algorithm to merge k sorted arrays into one sorted list, where n is the total number of elements in all the input lists.

Example: Assume the lists [1, 5, 7], [2, 4], [3, 6, 8, 9]. A three-way merge will then result in [1, 2, 3, 4, 5, 6, 7, 8, 9].

Hint: Use a min-heap containing the lists to be sorted. What should you use as sort key for each list in that heap?

Exercise 5 (3 points)

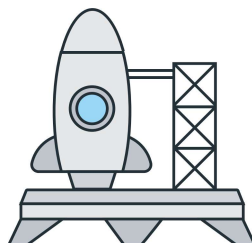
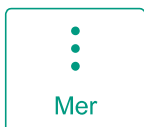
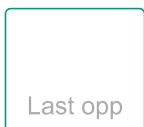
- a) What value of q does Partition (from the quicksort algorithm) return when all elements in the array $A[p \dots r]$ have the same value?
- b) What does this mean for the runtime of the algorithm?
- c) Modify Partition so that it returns $q = \lfloor (p + r)/2 \rfloor$ when all elements in the array $A[p \dots r]$ are equal.

Exercise 6 (3 points)

Imagine quicksort could always partition on the median of the data in each subarray $A[p \dots r]$.

- 1. What would this mean for the runtime of the algorithm?
- 2. If you think this can be implemented efficiently, give pseudocode for an algorithm that does this. Otherwise, argue why it cannot be implemented efficiently.

Velg innsendelsestype



Velg en fil å laste opp

Tillatt fil: PDF, IPYNB

eller

Canvas-filer



<https://nmbu.instructure.com/courses/13608/modules/items/387966>



<https://nmbu.instructure.com/courses/13608/modules/item>